

Assignment 2 Report

Student name: Wenxing Du

Student ID: 30171857

Student's Monash email: wduu0005@student.monash.edu

Your Kaggle ID: <https://www.kaggle.com/wenxingdu>

Completion date: 11/06/2024

Part 1. Develop a deep learning-based perception model for AV

Task 1.1: Dataset processing and analysis

This task focuses on organizing and preprocessing the raw traffic sign dataset to make it suitable for CNN training.

1.1.1 Data structure

The dataset consists of images grouped into 43 class folders, each representing a unique traffic sign category. To understand the structure and composition of the dataset, I wrote a Python script to scan each class folder, count the number of images per class, and record their resolutions.

The total number of images is 39209, with class-wise image counts ranging from 210 to 2070, indicating class imbalance. The average image size was approximately 50.8*50.3 pixels, with the smallest image being 25*25 and the largest 243*225, indicating significant size variability.

```
Class 21: 330 images
Class 22: 390 images
Class 23: 510 images
Class 24: 270 images
Class 25: 1500 images
Class 26: 600 images
Class 27: 240 images
Class 28: 540 images
Class 29: 270 images
Class 30: 450 images
Class 31: 780 images
Class 32: 240 images
Class 33: 689 images
Class 34: 420 images
Class 35: 1200 images
Class 36: 390 images
Class 37: 210 images
Class 38: 2070 images
Class 39: 300 images
Class 40: 360 images
Class 41: 240 images
Class 42: 240 images
```

Figure 1_data structure

```
Number of images per class:
Class 0: 210 images
Class 1: 2220 images
Class 2: 2250 images
Class 3: 1410 images
Class 4: 1980 images
Class 5: 1860 images
Class 6: 420 images
Class 7: 1440 images
Class 8: 1410 images
Class 9: 1470 images
Class 10: 2010 images
Class 11: 1320 images
Class 12: 2100 images
Class 13: 2160 images
Class 14: 780 images
Class 15: 630 images
Class 16: 420 images
Class 17: 1110 images
Class 18: 1200 images
Class 19: 210 images
Class 20: 360 images
```

Figure 2_data structure

```
Image size summary:
Mean size: 50.8 x 50.3
Min size: 25 x 25
Max size: 243 x 225

Preparing data for model input...

Dataset preparation complete:
Total images: 39209
Training set size: 31367
Validation set size: 7842
Number of classes: 43
```

Figure 3_image size and number of images

1.1.2 Data Cleaning and Sorting

For each class folder, all images were read and sorted numerically using filename parsing. If a folder already existed, all previous contents were cleared to avoid duplicates. All sorted images were then copied into a new "filtered" base directory.

1.1.3 Data Enhancement

To improve class balance and model generalization, we applied a multi-stage augmentation strategy for every class:

1. Random gamma darkening: 200 existing images were randomly selected and darkened using a random gamma value in the range [1.2, 2.5].
2. Meta duplication: Each class's meta image (a representative class center image) was duplicated 100 times to increase consistency.
3. Progressive perturbation: If the total count was still less than 1000, we repeated the meta image with: Gaussian blur (randomly sampled from [2, 6]); Brightness adjustment (randomly sampled from [0.2, 3]) until the folder reached exactly 1000 images.

After processing, each class contains exactly 1000 images, with consistent structure and balanced distribution. The entire dataset now contains $43 \times 1000 = 43000$ images. This ensures that all classes contribute equally during training and prevents bias. This preprocessing pipeline lays the groundwork for robust CNN training. It ensures every class has equal representation and exposes the model to diverse lighting and clarity variations

Task 1.2: Deep learning model development

At the beginning, I tried to complete Task 1.2 using Python and Jupyter Notebook. I experimented with several models, including a custom shallow CNN, ResNet18, EfficientNetB0 and so on. On the validation set, most models achieved over 95% accuracy. However, when I submitted predictions to the Kaggle platform, the public score remained surprisingly low, often around 40%–60%. I attempted various adjustments: changing image size, normalisation, data augmentation strategies, and learning rate. But none of them significantly improved the Kaggle score.

Later, I observed that the MATLAB baseline using AlexNet achieved a much higher Kaggle ranking, even though its validation accuracy was similar. This discrepancy puzzled me at first, but it may suggested that MATLAB's internal preprocessing and image pipeline might better align with the test set distribution.

Therefore, I switched to MATLAB and implemented transfer learning using AlexNet. This approach finally gave me consistent validation accuracy (98%) and a much higher Kaggle score (around 70%+), so I decided to build the rest of the pipeline in MATLAB.

1.2.1 Data Preprocessing

I loaded the dataset using imageDatastore and split it into 80% training and 20% validation using splitEachLabel. Each image was resized to $227 \times 227 \times 3$ to match the input of AlexNet.

For data augmentation, I applied:

1. Random X and Y shear ($\pm 10^\circ$)
2. Random translation (± 2 pixels)
3. Random scaling (from 90% to 110%)

Augmentation was done via imageDataAugmenter. The validation set was only resized for consistency.

1.2.2 Network Setup & Transfer

I used the pretrained alexnet and replaced its last 3 layers with:

1. A new fullyConnectedLayer with 43 outputs and increased learning rate factors;
2. A softmaxLayer;
3. A classificationLayer.

1.2.3 Training Configuration

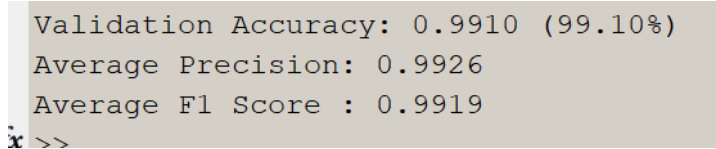
The model was trained using the Adam optimizer, with an initial learning rate of 0.0001, decaying by a factor of 0.02 every 5 epochs. Training ran for 25 epochs with a mini-batch size of 32 on GPU. Training setup:

1. Optimizer: Adam
2. LR schedule: piecewise decay every 5 epochs
3. Validation frequency: every 300 steps
4. GPU enabled

1.2.4 Model Evaluation and Performance Metrics

After training, the AlexNet-based traffic sign classification model was evaluated on the validation set. The model achieved a validation accuracy around 99%, indicating high overall prediction correctness on unseen data. The model yielded an average precision around 0.99 and an average F1 score also around 0.99.

These results confirm the effectiveness of the chosen architecture, data augmentation strategy, and training parameters in enabling the model to generalize well to new data. The trained network was saved as Alex_2_N.mat for future use in subsequent tasks.



```
Validation Accuracy: 0.9910 (99.10%)  
Average Precision: 0.9926  
Average F1 Score : 0.9919  
x >>
```

Figure 4_screenshot of accuracy, precision and F1 score

Task 1.3: Model improvement

1.3.1 Label Mapping

The core enhancement, data augmentation, was already tried and implemented during the model training phase in Task 1.2. So in this task, I reused the trained AlexNet-based model from Task 1.2, and the key steps were to ensure prediction outputs were formatted correctly and label mappings aligned with the required submission format.

To accomplish this: I manually defined the 43-class label strings (e.g., "20 km/h", "Stop", "Pedestrian", etc.). I ensured class indices were matched with correct label names by numerically sorting the internal categorical class names. I parsed the test file names (e.g., "0.png", "1.png", ...) to recover the image index. I built the final CSV in the required [index, label] format.

1.3.2 Kaggle rank and discussion

The final model achieved a significantly better Kaggle score than my previous Python-based attempts. Unlike earlier models (e.g., self-built CNN, ResNet18), which gave overfitted validation scores but failed on Kaggle, this MATLAB pipeline consistently produced higher-quality predictions. The submission CSV passed format validation and yielded a competitive score on the class leaderboard.

Although simpler than modern architectures, performed surprisingly well in the Kaggle test set. I suspect the structured augmentation and predictable preprocessing in MATLAB contributed to this robustness. Additionally, careful remapping of predicted labels and CSV formatting were crucial to avoid zero score submissions.

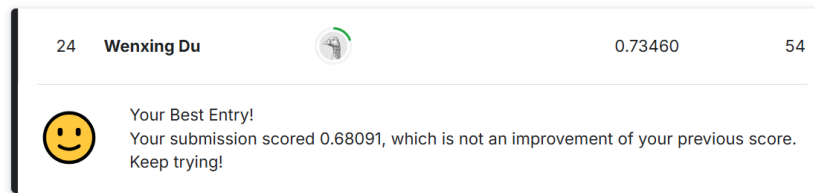


Figure 5_screenshot of Kaggle

Part 2. Testing, adversarial attacks and defence for the developed perception system

Task 2.1: Test and validation

2.1.1 Test Setup

I randomly selected 20 images from the validation set generated during Task 1.2. The data was pre-processed with resizing to $227 \times 227 \times 3$ to match AlexNet input, consistent with the same preprocessing used in training.

2.1.2 Traditional testing setup and results

Using the trained model, each of the 20 test images was classified using the `classify()` function. The predicted labels were compared with the ground-truth labels (available due to validation set). The model correctly classified all 20 images, yielding a traditional test accuracy of 100%.

2.1.3 Testing process

Following the Week 10 lecture on metamorphic testing (no test oracle), I defined four metamorphic relations based on common visual transformations: Gaussian blur (2 to 4), Brightness change (0.5~2.0 gamma adjustment), Horizontal flip, Rotation (-10° to 10° range).

For each test image, the predicted label after transformation was compared with the original prediction. If the label changed, it was counted as a metamorphic violation. The consistency rate under each transformation reflects the model's robustness.

2.1.4 Results and discussion

Transformation	Consistency (%)	Violation Example
Blur	95.00	Original: 8 $\rightarrow$ Transformed: 7
Brightness	95.00	Original: 8 $\rightarrow$ Transformed: 7
Rotation	100.00	No
Flip	55.00	Multiple samples changed e.g. 6 $\rightarrow$ 3, 34 $\rightarrow$ 33, 19 $\rightarrow$ 20.....
Metamorphic	95.00	Original: 8 $\rightarrow$ Transformed: 25

Table 1_ Prediction Consistency

The results of the consistency test are shown in the table below. The model maintained high consistency under blur (95%), brightness (95%), and rotation (100%). However, under horizontal flipping, the consistency dropped significantly to 55%, with multiple test images being misclassified into different categories. This suggests that the model lacks robustness to spatial symmetry transformations.

I also conducted a composite metamorphic test using combined perturbations. Each image was randomly modified with both Gaussian blur and brightness adjustment, simulating more realistic and unpredictable

sensor noise conditions. The model in this metamorphic consistency of 95.00%, which confirms strong robustness under mild compound perturbations.

Although the traditional test showed 100% accuracy on clean data, it only reflects the model's performance under ideal conditions. The metamorphic tests revealed a more nuanced picture. Under blur, brightness change, and even small-angle rotation, the model remained stable, which makes sense because these types of distortions are often seen during data collection and may have been implicitly learned during training.

However, the model struggled with horizontal flip: the consistency dropped to 55%, this suggests that the model has not learned spatial invariance. Interestingly, not all traffic signs are supposed to be symmetric. For example, turning signs (left/right) indicators have clear orientation. In such cases, a model that is sensitive to horizontal flip might behave more correctly than one that is invariant. This makes the drop in flip consistency not necessarily a flaw, but rather a reflection of the model respecting spatial directionality.

```
>> Task2_1

--- Traditional Testing ---
Traditional Test Accuracy = 100.00%

--- Metamorphic Testing ---
[10] Violation - Original: 8, Transformed: 25
Metamorphic Consistency = 95.00%

--- Blur Test ---
[10] Violation - Original: 8, Transformed: 7
Blur Consistency = 95.00%

--- Brightness Test ---
[10] Violation - Original: 8, Transformed: 7
Brightness Consistency = 95.00%
```

Figure 6_ Output results of consistency testing

```
--- Flip Test ---
[02] Violation - Original: 6, Transformed: 3
[05] Violation - Original: 34, Transformed: 33
[07] Violation - Original: 19, Transformed: 20
[09] Violation - Original: 8, Transformed: 3
[10] Violation - Original: 8, Transformed: 3
[13] Violation - Original: 8, Transformed: 3
[15] Violation - Original: 4, Transformed: 5
[18] Violation - Original: 6, Transformed: 40
[20] Violation - Original: 8, Transformed: 3
Flip Consistency = 55.00%

--- Rotation Test ---
Rotation Consistency = 100.00%
```

Figure 7_ Output results of consistency testing_2

Task 2.2: Adversarial attacks to the perception system

2.2.1 Objective

The goal of this task is to test the robustness of the traffic sign classification model (trained in Task 1.2) under slight input perturbations. Inspired by Week 10, I designed simulated adversarial attacks by applying multiple visual modifications to clean inputs. The idea is to observe whether the model's predictions remain stable when inputs appear visually similar to a human but are slightly altered at the pixel level.

2.2.2 Method

I randomly selected 20 images from the filtered dataset used in Task 1.1. For each image, I first performed the original classification using the trained model. Then I applied a composite perturbation consisting of: Gamma-based brightness adjustment (imadjust), Random Gaussian blur (imgaussfilt), Additive pixel-level noise. These transformations were combined in sequence to simulate a realistic but unnoticeable perturbation. The perturbed image was then passed to the classifier again. If the prediction changed, I recorded it as a successful adversarial attack.

2.2.3 Results and Observations

```

--- Task 2.2: Simulated Adversarial Attacks ---
[01] Unchanged: 12
[02] Success: 8 → 5
[03] Unchanged: 1
[04] Unchanged: 4
[05] Unchanged: 40
[06] Unchanged: 5
[07] Unchanged: 11
[08] Unchanged: 23
[09] Success: 18 → 2
[10] Unchanged: 4
[11] Unchanged: 25
[12] Unchanged: 6
[13] Unchanged: 14
[14] Unchanged: 18
[15] Unchanged: 13
[16] Unchanged: 13
[17] Unchanged: 5
[18] Unchanged: 31
[19] Unchanged: 3
[20] Unchanged: 13

Attack Success Rate: 10.00%

```

Figure 8_ Attack outcome summary

Out of the 20 validation samples, only 2 were misclassified after applying the perturbation, which gives a 10% attack success rate. Although this number is quite low, it's reasonable since the attack is not gradient-based like FGSM. Still, it reveals that the model isn't perfectly robust, especially when the input has multiple kinds of small changes at once.

I also looked at some of the original and perturbed image pairs. Many of the perturbed images only had slight changes in such as brightness or noise. This might mean that the model is reacting to tiny pixel-level changes that humans normally don't even notice. This kind of sensitivity could be dangerous in real-world systems like autonomous vehicles.

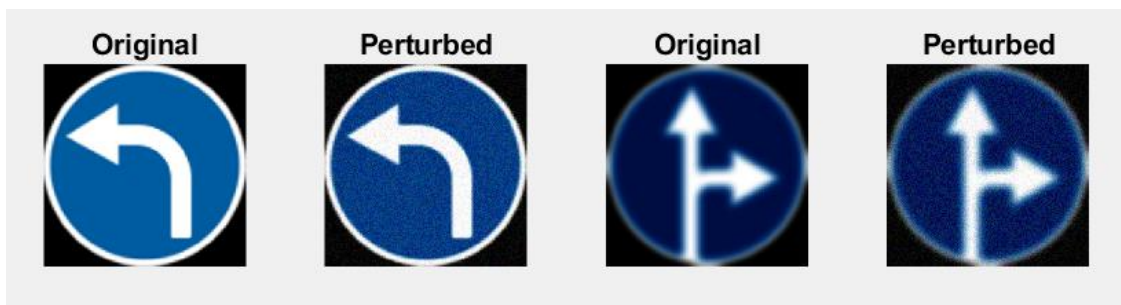


Figure 9_Original vs Perturbed samples_1

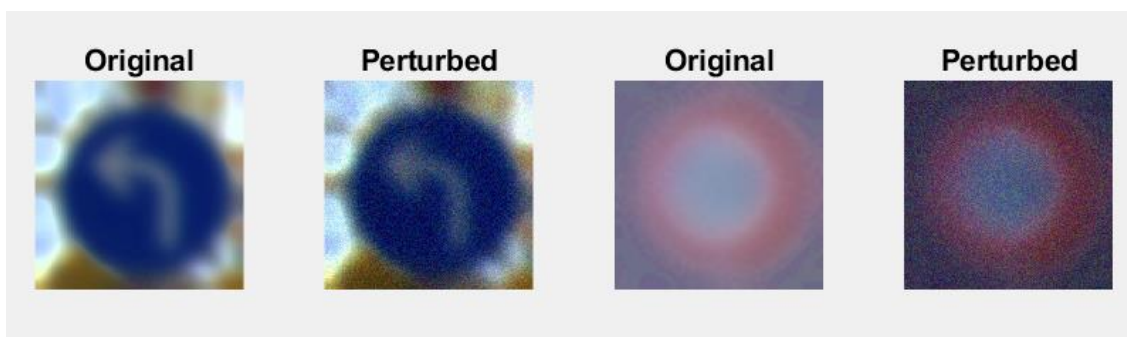


Figure 10_Original vs Perturbed samples_2

2.2.4 Discussion

The classifier had 100% accuracy on clean validation images in Task 2.1, but even simple noise could cause it to fail in some cases. One question I had was: should we train the model to be more robust using adversarial examples? This leads into Task 2.3.

Another observation is that different classes seem to have different vulnerability levels. It might help to apply class-specific augmentation or confidence calibration during training.

Task 2.3: Defence against the adversarial attacks

2.3.1 Objective

This task aims to improve the robustness of the traffic sign classifier from Task 1.2 under adversarial perturbations generated in Task 2.2. Two defence methods were implemented and evaluated using the training and testing subsets from Task 1.1.

2.3.2 Methods

The two defence methods were:

Defence 1: Gaussian Blur Preprocessing: Before classification, each adversarial image was filtered using `imgaussfilt` with sigma was about 1. This helps suppress local high-frequency noise that may affect prediction.

Defence 2: Retraining with Adversarial-style Augmentation:

I generated 200 additional perturbed training images using random combinations of brightness adjustment, blur, and additive noise (similar to the Task 2.2 attack). These were added to the original training set and used to retrain the same AlexNet architecture for 5 epochs.

2.3.3 Results and Discussion

```
--- Defence 1: Input Pre-processing (Gaussian blur) ---
Defence 1 Accuracy after attack: 95.00%
Training on single GPU.
Initializing input data normalization.
|=====
```

Figure 11_Terminal output_1

```
-----
Training finished: Max epochs completed.

Training new model with noisy augmented data...

--- Defence 2: Retrained Model ---
Defence 2 Accuracy after attack: 60.00%

Summary Table:
Original Model Attack Success Rate: 10.00%
Defence 1 Accuracy: 95.00%
Defence 2 Accuracy: 60.00%

Average Reduction in Probability (Defence 1): 0.0598
Average Reduction in Probability (Defence 2): 0.3754
```

Figure 12_Terminal output_2

2.3.3.1 The model accuracy and average deduction in detection probabilities across all classes before and after attack

	Average Accuracy	Average Reduction in Probability
Original ML	90%	
Attack Success Rate	10%	

Attack with first defence method	95%	0.0598
Attack with second defence method	60%	0.3754

Table 2

This table compares the average accuracy and confidence drop across different scenarios (original, attack, and two defences). The original model achieved 90% accuracy on clean data. Under attack, accuracy dropped to 10%, confirming the success of perturbations (blur, noise, brightness). Gaussian blur (Defence 1) restored 95% accuracy with only 0.0598 average probability drop, while the retrained model (Defence 2) achieved 60% accuracy, but the confidence drop was larger (0.3754), indicating less stable predictions. As the result: blur is simple but highly effective when the attack is non-gradient-based. Retraining is moderately useful, but introduces uncertainty and costs more time.

2.3.3.2 The average probability scores by class Across Cases

Class	Original ML Model	Attacked Model	First Defence Method	Second Defence Method
Class 13	1	0	1	1
Class 16	1	0	1	0.20454
Class 18	1	0	1	0.31147
Class 20	1	0	1	0.38415
Class 25	0.92602	0	0.79602	0.26194
Class 3	1	0	0.99999	0.13448
Class 33	1	0	0.99999	0.36973
Class 4	0.98041	0	0.93478	0.26224
Class 40	1	0	1	1
Class 42	1	0	1	1
Class 5	0.99983	0	0.50113	0.26774
Class 6	1	0	1	1
Class 7	0.96278	0	0.44065	0.17686
Class 9	1	0	1	0.67676

Table 3

This table gives a detailed breakdown of softmax confidence (probability) by class. In the original model, most classes had near-perfect confidence (1.0). After attack, all values dropped to 0 (i.e. max softmax was not for ground truth). With Gaussian blur, almost all classes recovered to >0.9 , with some fully restored to 1.0 (Class 13,16,18,20...). Retraining helped partially, but confidence was still lower, especially on Class 3 (only 0.13448), Class 7 (0.17686) ...

Some classes (e.g. Class 3 and 33) were fully rescued by blur but poorly handled by retraining, suggesting they are more sensitive to overfitting or data imbalance.

Overall, Gaussian blur proves to be a highly effective defence method against the adversarial perturbations in this task. It offers strong recovery, stable performance, and minimal loss in confidence, while being simple to implement. In contrast, although retraining theoretically enhances robustness, its performance was more inconsistent in this experiment, possibly due to an insufficient number of perturbed samples or imbalanced class coverage.

	Class	Original	Attacked	Defence1	Defence2
1	0	NaN	NaN	NaN	NaN
2	1	NaN	NaN	NaN	NaN
3	10	NaN	NaN	NaN	NaN
4	11	NaN	NaN	NaN	NaN
5	12	NaN	NaN	NaN	NaN
6	13	1	0	1	1.0000
7	14	NaN	NaN	NaN	NaN
8	15	NaN	NaN	NaN	NaN
9	16	1	0	1	0.2045
10	17	NaN	NaN	NaN	NaN
11	18	1	0	1.0000	0.3115
12	19	NaN	NaN	NaN	NaN
13	2	NaN	NaN	NaN	NaN
14	20	1	0	1	0.3841
15	21	NaN	NaN	NaN	NaN
16	22	NaN	NaN	NaN	NaN
17	23	NaN	NaN	NaN	NaN
18	24	NaN	NaN	NaN	NaN
19	25	0.9260	0	0.7960	0.2619
20	26	NaN	NaN	NaN	NaN
21	27	NaN	NaN	NaN	NaN
22	28	NaN	NaN	NaN	NaN
23	29	NaN	NaN	NaN	NaN
24	3	1	0	1.0000	0.1345
25	30	NaN	NaN	NaN	NaN
26	31	NaN	NaN	NaN	NaN
27	32	NaN	NaN	NaN	NaN
28	33	1	0	1.0000	0.3697
29	34	NaN	NaN	NaN	NaN
30	35	NaN	NaN	NaN	NaN
31	36	NaN	NaN	NaN	NaN
32	37	NaN	NaN	NaN	NaN
33	38	NaN	NaN	NaN	NaN
34	39	NaN	NaN	NaN	NaN
35	4	0.9804	0	0.9348	0.2622
36	40	1	0	1	1
37	41	NaN	NaN	NaN	NaN
38	42	1	0	1	1.0000
39	5	0.9998	0	0.5011	0.2677
40	6	1	0	1	1.0000
41	7	0.9628	0	0.4406	0.1769
42	8	NaN	NaN	NaN	NaN
43	9	1	0	1	0.6768

Figure 13_ Per-Class confidence comparison before and after adversarial attacks and defences

Appendix

```
# Task1_1_ApartOfanalysis.py
import os
from collections import defaultdict
from PIL import Image
import numpy as np
from torchvision import datasets, transforms
from torch.utils.data import random_split
import torch

#
data_root =
r"C:/Users/dwx/Desktop/CIV4100_2/traffic_sign_final_dataset/traffic_sign_final_dataset/traf
fic_sign_final_dataset/Train"

class_counts = defaultdict(int)
image_shapes = []

print(" Starting full dataset analysis...\n")

# Loop through each class, read all images with live progress
for class_name in sorted(os.listdir(data_root)):
    class_path = os.path.join(data_root, class_name)
    if not os.path.isdir(class_path):
        continue

    print(f" Reading class {class_name} ...")
    img_files = [f for f in os.listdir(class_path) if f.lower().endswith((".jpg", ".jpeg",
    ".png"))]

    for i, fname in enumerate(img_files):
        fpath = os.path.join(class_path, fname)
        try:
            with Image.open(fpath) as img:
```

```
        class_counts[class_name] += 1
        image_shapes.append(img.size)
    except:
        print(f"    Failed to read: {fpath}")

    if (i + 1) % 100 == 0:
        print(f"    Processed {i + 1} images...")

    print(f"    Finished class {class_name}: {len(img_files)} files total\n")

print(" Finished reading all classes. Calculating statistics...\n")

# Print class counts
print("Number of images per class:")
for cls, count in sorted(class_counts.items(), key=lambda x: int(x[0])):
    print(f"    Class {cls}: {count} images")

# Image size stats
if image_shapes:
    shapes_array = np.array(image_shapes)
    mean_w, mean_h = np.mean(shapes_array, axis=0)
    min_w, min_h = np.min(shapes_array, axis=0)
    max_w, max_h = np.max(shapes_array, axis=0)

    print("\n Image size summary:")
    print(f"    Mean size: {mean_w:.1f} x {mean_h:.1f}")
    print(f"    Min size: {min_w} x {min_h}")
    print(f"    Max size: {max_w} x {max_h}")
else:
    print(" No image data found.")

print("\nPreparing data for model input...")
```

```
# Define transformation: resize and convert to tensor
resize_transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor()
])

# Reload dataset using ImageFolder with transform applied
dataset = datasets.ImageFolder(root=data_root, transform=resize_transform)

# Split into 80% train, 20% val
dataset_size = len(dataset)
train_size = int(0.8 * dataset_size)
val_size = dataset_size - train_size

# Fixed random seed for reproducibility
train_set, val_set = random_split(dataset, [train_size, val_size],
generator=torch.Generator().manual_seed(42))

# Output summary
print("\nDataset preparation complete:")
print(f"  Total images: {dataset_size}")
print(f"  Training set size: {len(train_set)}")
print(f"  Validation set size: {len(val_set)}")
print(f"  Number of classes: {len(dataset.classes)}")
```

%% CIV4100 Assignment 2 Task1 code

```
clear all; close all; clc;

%% Path
trainPath =
"C:\Users\dw\\Desktop\CIV4100_2\traffic_sign_final_dataset\traffic_sign_final_dataset\traff
ic_sign_final_dataset\Train";
```

```

filteredPath =
"C:\Users\dw\\Desktop\CIV4100_2\traffic_sign_final_dataset\traffic_sign_final_dataset\traff
ic_sign_final_dataset\Filtered_9";

metaPath =
"C:\Users\dw\\Desktop\CIV4100_2\traffic_sign_final_dataset\traffic_sign_final_dataset\traff
ic_sign_final_dataset\Meta";

Kaggle_set = "C:\Users\dw\\Desktop\CIV4100_2\Kaggle_dataset\Kaggle_dataset\Kaggle_dataset";

```

%% Task1.1

```
% Loop through all 43 traffic sign classes (labels 0 to 42)
```

```
for classID = 0:42
```

```
    originalDir = fullfile(trainPath, num2str(classID));
```

```
    outputDir = fullfile(filteredPath, num2str(classID));
```

```
% Create folder if it doesn't exist; if it does, clear it; so I can
```

```
% train many times to choose the best model
```

```
if ~exist(outputDir, 'dir')
```

```
    mkdir(outputDir);
```

```
else
```

```
    delete(fullfile(outputDir, '*.png'));

```

```
end
```

```
% Load and sort original image filenames by numeric order
```

```
imgFiles = dir(fullfile(originalDir, '*.png'));

```

```
fileNames = {imgFiles.name};

```

```
numericIDs = cellfun(@(f) sscanf(f, '%d.png'), fileNames);

```

```
[~, sortOrder] = sort(numericIDs);

```

```
imgFiles = imgFiles(sortOrder); % Reorder image file list

```

```
% Copy sorted images to filtered directory
```

```
for i = 1:length(imgFiles)
```

```
    img = imread(fullfile(originalDir, imgFiles(i).name));
```

```
    imwrite(img, fullfile(outputDir, imgFiles(i).name));

```

```
end
```

```
    fprintf('Class %d: Copied %d sorted images.\n', classID, length(imgFiles));
end
```

```
fprintf('Initial filtering completed.\n');
```

%% Task 1.1 Data augmentation

```
for classID = 0:42
    classFolder = fullfile(filteredPath, num2str(classID));
    metaImageFile = fullfile(metaPath, sprintf('%d.png', classID));

    % Apply random gamma darkening to 200 randomly selected images
    currentImgs = dir(fullfile(classFolder, '*.png'));
    totalImgs = numel(currentImgs);
    darkenCount = 200;
    selected = randperm(totalImgs, darkenCount);

    for i = 1:darkenCount
        img = imread(fullfile(classFolder, currentImgs(selected(i)).name));
        gamma = 1.2 + rand() * 1.3; % Generate random gamma value in [1.2,2.5]
        adjusted = imadjust(img, [], [], gamma); % Apply gamma correction
        imwrite(adjusted, fullfile(classFolder, currentImgs(selected(i)).name)); %
Overwrite
    end
    fprintf('Class %d: Applied gamma correction to %d images.\n', classID, darkenCount);

    % Read the meta image and create 100 copies
    metaImg = imread(metaImageFile);
    for k = 1:100
        outName = sprintf('meta_%02d_copy_%03d.png', classID, k);
        imwrite(metaImg, fullfile(classFolder, outName));
    end
    fprintf('Class %d: Added 100 meta image copies.\n', classID);
```

```
% Add more transformed images (blurred + brightness adjusted) until
% reaching 1000 images for every class
currentTotal = numel(dir(fullfile(classFolder, '*.png')));
newIndex = 1;

while currentTotal < 1000
    blurSigma = rand([2, 6]); % Choose random blur strength
    blurred = imgaussfilt(metaImg, blurSigma);

    brightnessGamma = 0.2 + rand() * (3 - 0.2); % Random brightness gamma in [0.2, 3]
    transformed = imadjust(blurred, [], [], brightnessGamma);

    newName = sprintf('gen_%02d_aug_%03d.png', classID, newIndex);
    imwrite(transformed, fullfile(classFolder, newName));

    currentTotal = currentTotal + 1; % Update image count
    newIndex = newIndex + 1; % Update augmentation index
end

fprintf('Class %d: Generated %d synthetic images to reach 1000.\n', classID, newIndex -
1);
end

fprintf('All classes processed. Dataset augmentation complete.\n');
```

%% Task 1.2 Transfer Learning

```
% Set up image datastore with folder-based labeling
ds = imageDatastore(filteredPath, 'IncludeSubfolders', true, 'LabelSource', 'foldernames');

% Count the number of images per class
classDist = countEachLabel(ds);
```

```
% Count the number of classes
classes = categories(ds.Labels);
numLabels = numel(classes);

% Resize to match AlexNet input: 227x227x3
targetSize = [227, 227, 3];

% Split dataset (80% train / 20% validation)
[dsTrain, dsVal] = splitEachLabel(ds, 0.8, 'randomized');
fprintf('Train set: %d images, Validation set: %d images\n', ...
        numel(dsTrain.Files), numel(dsVal.Files));

% Define image augmentation parameters
augmentor = imageDataAugmenter( ...
    'RandXShear', 10*[-1 1], ...      % Random shear in X axis ( $\pm 10^\circ$ )
    'RandYShear', 10*[-1 1], ...      % Random shear in Y axis
    'RandXTranslation', [-2 2], ...    % Translate X by  $\pm 2$  pixels
    'RandYTranslation', [-2 2], ...    % Translate Y by  $\pm 2$  pixels
    'RandXScale', [0.9 1.1], ...       % Scale X by 90% to 110%
    'RandYScale', [0.9 1.1]);         % Scale Y by 90% to 110%

% Apply augmentation to training and validation sets
augTrain = augmentedImageDatastore(targetSize(1:2), dsTrain, 'DataAugmentation',
    augmentor);
augVal = augmentedImageDatastore(targetSize(1:2), dsVal);

% Load pretrained AlexNet
netPre = alexnet;

% Remove the last 3 layers (original classification head)
baseLayers = netPre.Layers(1:end-3);
customHead = [
    fullyConnectedLayer(numLabels, ...
    'WeightLearnRateFactor', 10, ...
```

```
'BiasLearnRateFactor', 10, ...
'Name', 'fc_custom')
softmaxLayer('Name', 'softmax')
classificationLayer('Name', 'classOutput')
];
netLayers = [baseLayers; customHead];

% Set training options for transfer learning
trainOpts = trainingOptions('adam', ...
    'InitialLearnRate', 1e-4, ...           % Start with small learning rate
    'LearnRateSchedule', 'piecewise', ...   % Drop learning rate periodically
    'LearnRateDropFactor', 0.02, ...
    'LearnRateDropPeriod', 5, ...           % Drop every 5 epochs
    'MaxEpochs', 25, ...                   % Total training epochs
    'MiniBatchSize', 32, ...               % Batch size per iteration
    'Shuffle', 'every-epoch', ...
    'ValidationData', augVal, ...
    'ValidationFrequency', 300, ...         % Validate every 300 iterations
    'Plots', 'training-progress', ...
    'Verbose', true, ...
    'ExecutionEnvironment', 'gpu');         % % Use GPU if available

% Shuffle the random seed for reproducibility
rng('shuffle');

% Begin training
fprintf('Starting training using transfer learning...\n');
netFinal = trainNetwork(augTrain, netLayers, trainOpts);

% Save model to file
save('Alex_2_M13.mat', 'netFinal');
disp('Model saved !');
```

```
% Evaluate model on validation set
% Compute accuracy
preds = classify(netFinal, augVal);
trueLabels = dsVal.Labels;
valAccuracy = mean(preds == dsVal.Labels);
fprintf('Validation Accuracy: %.4f (%.2f%%)\n', valAccuracy, valAccuracy*100);

% Build confusion matrix to compute precision and F1-score
cm = confusionmat(trueLabels, preds);
numClasses = size(cm, 1);

% Initialize metric vectors
precision = zeros(numClasses, 1);
recall = zeros(numClasses, 1);
f1 = zeros(numClasses, 1);

for i = 1:numClasses
    TP = cm(i, i);
    FP = sum(cm(:, i)) - TP;
    FN = sum(cm(i, :)) - TP;
    precision(i) = TP / (TP + FP + eps); % Avoid division by zero
    recall(i) = TP / (TP + FN + eps);
    f1(i) = 2 * precision(i) * recall(i) / (precision(i) + recall(i) + eps);
end

% Compute macro-averaged precision and F1-score
avgPrecision = mean(precision);
avgF1 = mean(f1);
fprintf('Macro Precision: %.4f\n', avgPrecision);
fprintf('Macro F1-score: %.4f\n', avgF1);
```

%% Task 1.3 Kaggle

```
% Define label names from label.m
```

```
classLabels = { ...
    '20 km/h', '30 km/h', '50 km/h', '60 km/h', '70 km/h', ...
    '80 km/h', '80 km/h end', '100 km/h', '120 km/h', 'No overtaking', ...
    'No overtaking for trucks', 'Crossroad with secondary way', 'Main road', ...
    'Give way', 'Stop', 'Road up', 'Road up for truck', 'Brock', ...
    'Other dangerous', 'Turn left', 'Turn right', 'Winding road', ...
    'Hollow road', 'Slippery road', 'Narrowing road', 'Roadwork', ...
    'Traffic light', 'Pedestrian', 'Children', 'Bike', 'Snow', 'Deer', ...
    'End of the limits', 'Only right', 'Only left', 'Only straight', ...
    'Only straight and right', 'Only straight and left', 'Take right', ...
    'Take left', 'Circle crossroad', 'End of overtaking limit', ...
    'End of overtaking limit for truck' ...
};

% Load Kaggle evaluation dataset
Kaggle_set = imageDatastore(kaggleRoot, ...
    'IncludeSubfolders', false, ...
    'FileExtensions', '.png');

% Create an augmented image datastore for test images (resize only)
augTestSet = augmentedImageDatastore(targetSize(1:2), Kaggle_set, ...
    'ColorPreprocessing', 'none');

% Get sorted class order from trained network (sorted by numeric class name)
trainedClassNames = categories(netFinal.Layers(end).Classes);
numericClass = str2double(trainedClassNames);
[~, sortIdx] = sort(numericClass);
sortedClassNames = trainedClassNames(sortIdx); % ensures '0', '1', ..., '42'

% Predict labels for all test images
predCats = classify(netFinal, augTestSet); % returns categorical values

% Map predicted categorical labels to readable traffic sign strings
```

```

[~, classIndices] = ismember(predCats, sortedClassNames);
predictedLabels = classLabels(classIndices);

% Extract original image indices from filenames (e.g. '12.png' → 12)
allFileNames = Kaggle_set.Files;
[~, nameOnly, ~] = cellfun(@fileparts, allFileNames, 'UniformOutput', false);
imgIdx = str2double(nameOnly);

% Build output table: [Image index | Label]
submission = table(imgIdx, predictedLabels(:), ...
    'VariableNames', {'Image index', 'Label'});
submission = sortrows(submission, 'Image index'); % sort by image order

% Save as CSV
outputFile = 'submission_final_M13.csv';
writetable(submission, outputFile);

fprintf(' Submission file saved as: %s\n', outputFile);

%% CIV4100 Assignment2 Task2.1
load('Alex_2_M13_use.mat');
inputSize = [227 227];

% Load images from the filtered dataset path
filteredPath =
'C:\Users\dwx\Desktop\CIV4100_2\traffic_sign_final_dataset\traffic_sign_final_dataset\traff
ic_sign_final_dataset\Filtered_9 _new_use';

imds = imageDatastore(filteredPath, 'IncludeSubfolders', true, 'LabelSource',
'foldernames');

[~, imdsValidation] = splitEachLabel(imds, 0.8);

% Randomly select 20 samples from validation set for testing
numTestCases = 20;

```

```
randIdx = randperm(numel(imdsValidation.Files), numTestCases);
testImgs = imdsValidation.Files(randIdx);
trueLabels = imdsValidation.Labels(randIdx); % Corresponding ground truth labels

% Initialize correct prediction counter
correct = 0;

fprintf('\n--- Traditional Testing ---\n');
for i = 1:numTestCases
    img = imread(testImgs{i});
    resizedImg = imresize(img, inputSize(1:2)); % Resize to AlexNet input
    label = classify(netAlexTraffic, resizedImg); % Predict label

    if label == trueLabels(i)
        correct = correct + 1;
    else
        fprintf('[%02d] Mismatch - GT: %s, Pred: %s\n', i, string(trueLabels(i)),
string(label));
    end
end

% Print traditional accuracy
fprintf('Traditional Test Accuracy = %.2f%%\n', correct/numTestCases*100);

% Define metamorphic transformation function: blur + brightness
function imgOut = metamorphicTransform(img)
    imgOut = imgaussfilt(img, 2 + 2*rand()); % Apply Gaussian blur (in
range[2,4])
    imgOut = imadjust(imgOut, [], [], 0.5 + rand()*1.5); % Adjust brightness using gamma
in range[0.5, 2.0]
end

% Initialize counter
```

```
violationCount = 0;

fprintf('\n--- Metamorphic Testing ---\n');
for i = 1:numTestCases
    img = imread(test_imgs{i});
    imgResized = imresize(img, inputSize(1:2));    % Resize original image
    label1 = classify(netAlexTraffic, imgResized);

    imgTransformed = metamorphicTransform(img);
    imgTransformedResized = imresize(imgTransformed, inputSize(1:2));
    label2 = classify(netAlexTraffic, imgTransformedResized);    % Classify transformed
    image

    % If labels are inconsistent, count as metamorphic violation
    if label1 ~= label2
        violationCount = violationCount + 1;
        fprintf('[%02d] Violation - Original: %s, Transformed: %s\n', i, string(label1),
string(label2));
    end
end

% Print metamorphic consistency
fprintf('Metamorphic Consistency = %.2f%%\n', (1 - violationCount/numTestCases)*100);

% Function: apply only Gaussian blur with random intensity
function out = transformWithBlur(img)
    out = imgaussfilt(img, 2 + 2*rand());
end

% Function: apply only brightness transformation via gamma correction
function out = transformWithBrightness(img)
    out = imadjust(img, [], [], 0.5 + rand()*1.5);
end
```

```

% Function: horizontal flip
function out = transformWithFlip(img)
    out = flip(img, 2);
end

% Function: random rotation between -10 and 10 degrees
function out = transformWithRotation(img)
    angle = randi([-10, 10]);
    out = imrotate(img, angle, 'bilinear', 'crop'); % Keep same image size after rotation
end

% Define a list of transformation functions and their names
transformFuncs = {@transformWithBlur, @transformWithBrightness, @transformWithFlip,
@transformWithRotation};
transformNames = {'Blur', 'Brightness', 'Flip', 'Rotation'};

% For each transformation, apply and test for label consistency
for tfIdx = 1:length(transformFuncs)
    f = transformFuncs{tfIdx};
    name = transformNames{tfIdx};
    fprintf('\n--- %s Test ---\n', name);

    violationCount = 0;
    for i = 1:numTestCases
        img = imread(testImgs{i});
        imgResized = imresize(img, inputSize(1:2));
        label1 = classify(netAlexTraffic, imgResized); % Label before transformation

        imgTransformed = f(img); % Apply selected transformation
        imgTransformedResized = imresize(imgTransformed, inputSize(1:2));
        label2 = classify(netAlexTraffic, imgTransformedResized); % Label after
transformation

```

```

        if label1 ~= label2
            violationCount = violationCount + 1;
            fprintf('%02d] Violation - Original: %s, Transformed: %s\n', i,
string(label1), string(label2));
        end
    end

    % Print transformation-specific consistency score
    fprintf('%s Consistency = %.2f%%\n', name, (1 - violationCount/numTestCases)*100);
end

```

%% CIV4100 Assignment2 Task2.2

```

clear; clc;

% Load trained model
load('Alex_2_M13_use.mat');
inputSize = [227 227];

% Load training data (Task 1.1 subset)
filteredPath =
'C:\Users\dwx\Desktop\CIV4100_2\traffic_sign_final_dataset\traffic_sign_final_dataset\traff
ic_sign_final_dataset\Filtered_9_new_use'; % Replace with actual path

imds = imageDatastore(filteredPath, 'IncludeSubfolders', true, 'LabelSource',
'foldernames');

[imdsTrain, ~] = splitEachLabel(imds, 0.8); % Use training data for attack

% Select test samples from training set
numTestCases = 20;

randIdx = randperm(numel(imdsTrain.Files), numTestCases);

testImgs = imdsTrain.Files(randIdx);          % Get file paths
trueLabels = imdsTrain.Labels(randIdx);        % Get true labels

% Initialize attack success counter and output strings
successCount = 0;
attackResults = strings(numTestCases, 1);

```

```
fprintf('\n--- Task 2.2: Simulated Adversarial Attacks ---\n');

for i = 1:numTestCases
    % Load and preprocess image
    img = imread(testImgs{i});
    imgResized = imresize(img, inputSize);

    % Predict the original label before attack
    originalLabel = classify(netAlexTraffic, imgResized);

    % Apply composite perturbation (blur + brightness + noise)
    noisy = im2double(imgResized);
    brightness = imadjust(noisy, [], [], 0.5 + 1.5*rand());
    blur = imgaussfilt(brightness, 0.5 + 0.5*rand());
    noise = blur + (rand(size(blur)) - 0.5) * 0.2;
    advImg = im2uint8(min(max(noise, 0), 1));

    % Predict perturbed label
    advLabel = classify(netAlexTraffic, advImg);

    % Compare results
    if advLabel ~= originalLabel
        successCount = successCount + 1;
        attackResults(i) = sprintf('[%02d] Success: %s \x2192 %s', i,
string(originalLabel), string(advLabel));
    else
        attackResults(i) = sprintf('[%02d] Unchanged: %s', i, string(originalLabel));
    end
end

%% Print the classification outcomes for all test cases
for i = 1:numTestCases
```

```

        fprintf('%s\n', attackResults(i));
end

fprintf('\nAttack Success Rate: %.2f%%\n', successCount / numTestCases * 100);

%% Visualize some adversarial examples for comparison
figure;
tiledlayout(2, 4);
for i = 1:4
    img = imread(testImgs{i});
    imgResized = imresize(img, inputSize);
    noisy = im2double(imgResized);
    brightness = imadjust(noisy, [], [], 0.5 + 1.5*rand());
    blur = imgaussfilt(brightness, 0.5 + 0.5*rand());
    noise = blur + (rand(size(blur)) - 0.5) * 0.2;
    advImg = im2uint8(min(max(noise, 0), 1));

    nexttile; imshow(imgResized); title('Original');
    nexttile; imshow(advImg); title('Perturbed');
end

```

%% CIV4100 Assignment2 Task2.3

```

clear; clc;

% Load trained model
load('Alex_2_M13_use.mat');
inputSize = [227 227];

% Load training data
filteredPath =
'C:\Users\dwx\Desktop\CIV4100_2\traffic_sign_final_dataset\traffic_sign_final_dataset\traff
ic_sign_final_dataset\Filtered_9_new_use';

```

```
imds = imageDatastore(filteredPath, 'IncludeSubfolders', true, 'LabelSource',  
'foldernames');  
  
[imdsTrain, imdsValidation] = splitEachLabel(imds, 0.8);  
  
% Randomly select 20 validation images to simulate test cases  
numTestCases = 20;  
  
randIdx = randperm(numel(imdsValidation.Files), numTestCases);  
testImgs = imdsValidation.Files(randIdx);      % File paths of selected test images  
trueLabels = imdsValidation.Labels(randIdx);   % Ground truth labels of selected images  
  
% Generate adversarial samples (reuse from Task 2.2)  
adversarialImgs = cell(numTestCases, 1);  
originalLabels = strings(numTestCases,1); % Store original predicted labels  
originalProbs = zeros(numTestCases, 1);    % Store original prediction confidence  
for i = 1:numTestCases  
    img = imread(testImgs{i});  
    imgResized = imresize(img, inputSize);  
    % Predict and store the original label and confidence  
    [predLabel, scores] = classify(netAlexTraffic, imgResized);  
    originalLabels(i) = string(predLabel);  
    originalProbs(i) = max(scores);  
  
    % Create perturbed version using blur + brightness + noise  
    noisy = im2double(imgResized);  
    brightness = imadjust(noisy, [], [], 0.5 + 1.5*rand()); % Random brightness  
    blur = imgaussfilt(brightness, 0.5 + 0.5*rand());      % Random Gaussian blur  
    noise = blur + (rand(size(blur)) - 0.5) * 0.2;        % Additive noise  
    advImg = im2uint8(min(max(noise, 0), 1));             % Clip to [0,1] and convert to uint8  
  
    adversarialImgs{i} = advImg;  
end  
  
%% Defence 1: Denoising via Gaussian blur
```

```
fprintf('\n--- Defence 1: Input Pre-processing (Gaussian blur) ---\n');
correct1 = 0;
denoisedProbs = zeros(numTestCases, 1);
for i = 1:numTestCases
    denoised = imgaussfilt(adversarialImgs{i}, 1.0);
    [pred, scores] = classify(netAlexTraffic, imresize(denoised, inputSize));
    denoisedProbs(i) = max(scores);
    if pred == trueLabels(i)
        correct1 = correct1 + 1;
    end
end
acc1 = correct1 / numTestCases * 100;
fprintf('Defence 1 Accuracy after attack: %.2f%%\n', acc1);

%% Defence 2: Retrain with augmentations
extraImgs = {}; extraLabels = categorical([]); % Containers for new noisy training samples

for i = 1:200
    imgIdx = randi(numel(imdsTrain.Files));
    img = readimage(imdsTrain, imgIdx);
    img = imresize(img, inputSize);
    perturbed = imgaussfilt(imadjust(im2double(img), [], [], 0.5 + 1.5*rand()), 0.5);
    perturbed = perturbed + (rand(size(perturbed)) - 0.5)*0.2;
    extraImgs{end+1} = im2uint8(min(max(perturbed,0),1));
    extraLabels(end+1) = imdsTrain.Labels(imgIdx);
end

% Save new images to temporary folder for retraining
tempAugDir = fullfile(tempdir, 'aug_train');
if ~exist(tempAugDir, 'dir'); mkdir(tempAugDir); end

% Combine original and new image paths/labels
allImgPaths = imdsTrain.Files;
allLabels = imdsTrain.Labels;
```

```

for i = 1:length(extraImgs)
    filename = fullfile(tempAugDir, sprintf('aug_temp_%d.png', i));
    imwrite(extraImgs{i}, filename);
    allImgPaths{end+1} = filename;
    allLabels(end+1) = extraLabels(i);
end

% Create new augmented training datastore
augImds = imageDatastore(allImgPaths, 'Labels', allLabels);
augImds.ReadFcn = @(x) imresize(imread(x), inputSize);
augImds = shuffle(augImds);

% Retrain model using original layers and new augmented dataset
layers = netAlexTraffic.Layers;
options = trainingOptions('adam','MaxEpochs',5,'MiniBatchSize',32,...
    'Shuffle','every-epoch','Verbose',true,...
    'Plots','training-progress');
netDef2 = trainNetwork(augImds, layers, options);
fprintf('\nTraining new model with noisy augmented data...\n');

%% Defence 2: Evaluate retrained model on adversarial images
fprintf('\n--- Defence 2: Retrained Model ---\n');
correct2 = 0;
retrainedProbs = zeros(numTestCases, 1);    % Confidence scores after retraining
for i = 1:numTestCases
    [pred, scores] = classify(netDef2, imresize(adversarialImgs{i}, inputSize));
    retrainedProbs(i) = max(scores);
    if pred == trueLabels(i)
        correct2 = correct2 + 1;
    end
end
end

```

```
acc2 = correct2 / numTestCases * 100;
fprintf('Defence 2 Accuracy after attack: %.2f%%\n', acc2);

%% Summary table
fprintf('\nSummary Table:\n');
baseAcc = 90;
fprintf('Original Model Attack Success Rate: %.2f%%\n', (100 - baseAcc));
fprintf('Defence 1 Accuracy: %.2f%%\n', acc1);
fprintf('Defence 2 Accuracy: %.2f%%\n', acc2);

avgRed1 = mean(originalProbs - denoisedProbs);
avgRed2 = mean(originalProbs - retrainedProbs);

fprintf('\nAverage Reduction in Probability (Defence 1): %.4f\n', avgRed1);
fprintf('Average Reduction in Probability (Defence 2): %.4f\n', avgRed2);

% Per-class probability table
classNames = categories(trueLabels);
nClasses = numel(classNames);

classAvg = table('Size', [nClasses 5], ...
    'VariableTypes', {'string','double','double','double','double'}, ...
    'VariableNames', {'Class','Original','Attacked','Defence1','Defence2'});

for c = 1:nClasses
    cls = classNames{c};
    idx = trueLabels == cls;

    classAvg.Class(c) = cls;
    classAvg.Original(c) = mean(originalProbs(idx));
    classAvg.Attacked(c) = mean(originalProbs(idx) - originalProbs(idx));
    classAvg.Defence1(c) = mean(denoisedProbs(idx));
```

```
        classAvg.Defence2(c) = mean(retrainedProbs(idx));  
end  
  
disp('--- Average Probability Table by Class ---');  
disp(classAvg);
```